

**Department of Computer Engineering COMSATS University  
Islamabad, Attock Campus**

| <b>Lab Rubrics Form</b> |                                                               |
|-------------------------|---------------------------------------------------------------|
| <b>Lab Name</b>         | Analysis of Discrete-Time Signal Downsampling and Upsampling. |
| <b>Lab Number</b>       | 6                                                             |
| <b>Submitted by:</b>    |                                                               |
| <b>Student Names</b>    | <b>Registration #</b>                                         |
| Sumaira Safeer          | FA22-BCE-019                                                  |

| Rubrics                           |                                                                                                                 | Marks |   |   |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------|-------|---|---|
|                                   |                                                                                                                 | 1     | 2 | 3 |
| PLO-5<br>Modern Tools<br>Usage    | Ability to execute the experiment utilizing hardware, software tools, or a combination of both.                 |       |   |   |
| PLO-10<br>Communication<br>Skills | Present lab activities effectively through detailed lab reports, oral examinations, and engaging presentations. |       |   |   |

## Lab-06

### Discrete-Time Signal Sampling: Downsampling and Upsampling Techniques

#### Objective:

The objective of this lab was to understand and implement two fundamental discrete-time signal sampling techniques: downsampling and upsampling. Specifically, the tasks included:

1. **Downsampling** by a factor of  $M=2$  to reduce sampling rate by selecting every  $M$ -th sample.
2. **Upsampling** by a factor of  $L=3$  (inserting 2 samples between each original sample) using linear interpolation to increase the sampling rate.

#### Lab Task:

Given the discrete-time signal

$$x[n] = \{2, 4, 6, 8, 10\}, \quad 0 \leq n \leq 4$$

1. Plot the original signal using a stem plot.
2. Downsample the signal by a factor of 2.
3. Plot and compare both signals on the same axis.

#### Code:

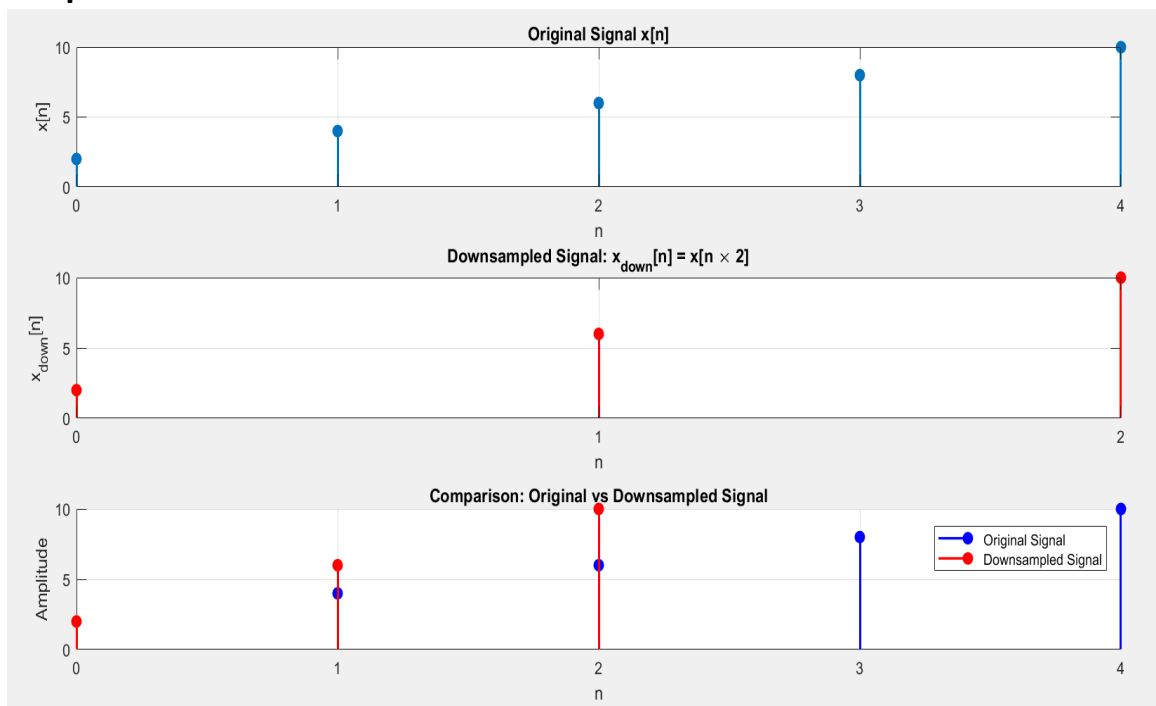
```
clc; clear all; close all;
% Original signal
n = 0:4;
x = [2 4 6 8 10];
% Downsample factor
M = 2;
% Downsample using x_down[n] = x[n * M]
n_down = 0:2; % [0, 1, 2]
x_down = x(1:M:end); % [2, 6, 10]
% 1. Plot original signal
figure;
subplot(3,1,1);
stem(n, x, 'filled', 'LineWidth', 1.5);
title('Original Signal x[n]');
xlabel('n'); ylabel('x[n]');
grid on;
xticks(n); % Only show integer values
% 2. Plot downsampled signal
subplot(3,1,2);
```

```

stem(n_down, x_down, 'r', 'filled', 'LineWidth', 1.5);
title('Downsampled Signal:  $x_{\text{down}}[n] = x[n \times 2]$ ');
xlabel('n'); ylabel('x_{down}[n]');
grid on;
xticks(n_down); % Only show integer values
% 3. Plot and compare both signals on same axis
subplot(3,1,3);
hold on;
% Original signal (blue)
stem(n, x, 'b', 'filled', 'LineWidth', 1.5);
% Downsampled signal (red)
stem(n_down, x_down, 'r', 'filled', 'LineWidth', 1.5);
hold off;
grid on;
xlabel('n');
ylabel('Amplitude');
title('Comparison: Original vs Downsampled Signal');
legend('Original Signal', 'Downsampled Signal');
xticks(0:4); % Only show integer values from 0 to 4

```

## Output:



## Lab Task 02:

Given the discrete-time signal

$$x[n] = \{2, 4, 6, 8, 10\}, \quad 0 \leq n \leq 4$$

1. Plot the original signal using a stem plot.
2. Upsample the signal by a factor of 2 by inserting two sample between the samples using linear interpolation.
3. Plot and compare both signals on the same axis.

### Code:

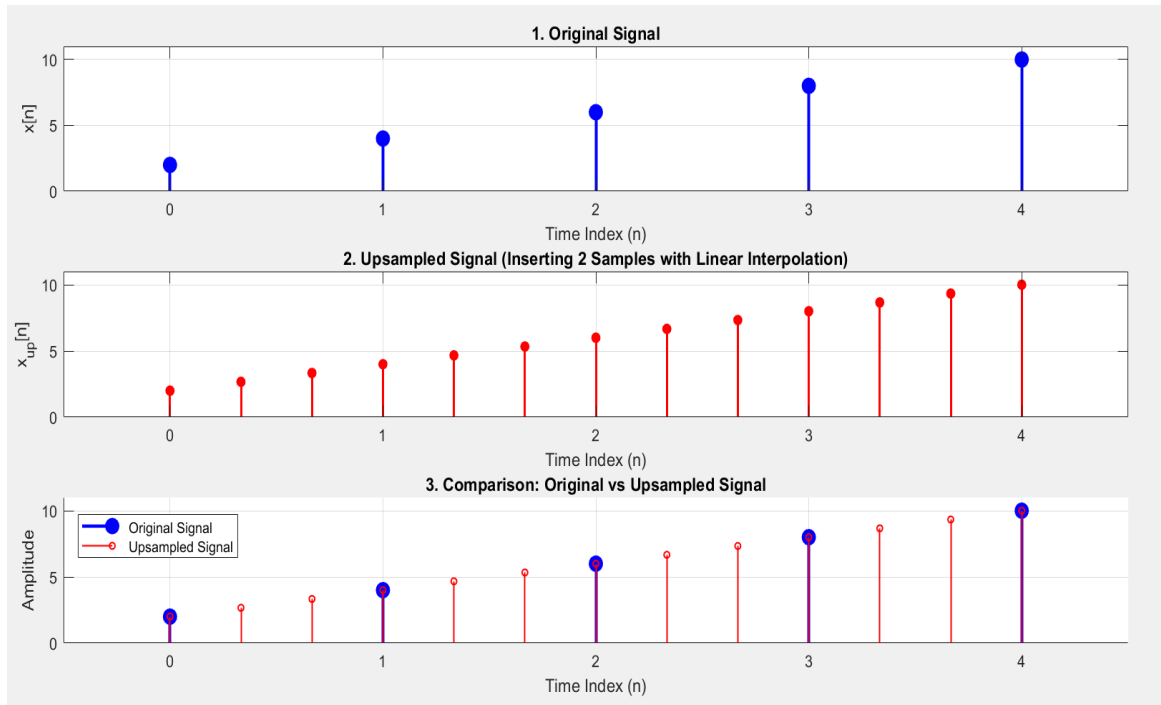
```
clc; clear all; close all;
% Original signal
n_original = 0:4;
x_original = [2, 4, 6, 8, 10];
L = 3; % This gives 2 new samples between each original
figure; % 1. Plot original signal
subplot(3,1,1);
stem(n_original, x_original, 'b', 'filled', 'LineWidth', 2, 'MarkerSize', 8);
title('1. Original Signal');
xlabel('Time Index (n)');
ylabel('x[n]');
grid on;
xlim([-0.5, 4.5]);
ylim([0, 11]);
xticks(0:4);
n_upsampled = 0:1/3:4; % Step size = 1/3 to insert 2 samples between integers
% Perform linear interpolation
x_upsampled = interp1(n_original, x_original, n_upsampled, 'linear');
% Display the results
fprintf('Original signal:\n');
fprintf('n = '); fprintf('%d ', n_original); fprintf('\n');
fprintf('x = '); fprintf('%d ', x_original); fprintf('\n\n');
fprintf('Upsampled signal (inserting 2 samples between each):\n');
fprintf('n = '); fprintf('%2f ', n_upsampled); fprintf('\n');
fprintf('x = '); fprintf('%2f ', x_upsampled); fprintf('\n\n');
% Plot upsampled signal
subplot(3,1,2);
stem(n_upsampled, x_upsampled, 'r', 'filled', 'LineWidth', 1.5, 'MarkerSize', 5);
```

```

title('2. Upsampled Signal (Inserting 2 Samples with Linear Interpolation)');
xlabel('Time Index (n)');
ylabel('x_{up}[n]');
grid on;
xlim([-0.5, 4.5]);
ylim([0, 11]);
xticks(0:4);
% 3. Plot and compare both signals on the same axis
subplot(3,1,3);
hold on;
% Plot original signal (thicker stems)
stem(n_original, x_original, 'b', 'filled', 'LineWidth', 2, 'MarkerSize', 8);
% Plot upsampled signal (thinner stems)
stem(n_upsampled, x_upsampled, 'r', 'LineWidth', 1, 'MarkerSize', 4);
hold off;
title('3. Comparison: Original vs Upsampled Signal');
xlabel('Time Index (n)');
ylabel('Amplitude');
legend('Original Signal', 'Upsampled Signal', 'Location', 'northwest');
grid on;
xlim([-0.5, 4.5]);
ylim([0, 11]);
xticks(0:4);

```

### Output:



### Conclusion:

In this lab, we successfully implemented downsampling and upsampling operations on a given discrete-time signal  $x[n]=[2,4,6,8,10]$  for  $n=0$  to  $4$ .

- **Downsampling ( $M=2$ )** resulted in  $x_{down}[n]=[2,6,10]$ , effectively reducing the number of samples by half while preserving the original amplitude values at even indices.
- **Upsampling ( $L=3$ )** introduced two new samples between each original sample using linear interpolation, producing a smoother and more densely sampled signal  $x_{up}[n]$ .

The lab demonstrated how downsampling reduces data points and sampling rate, while upsampling increases resolution through interpolation. Both operations are essential in digital signal processing for applications such as audio processing, image resizing, and communication systems. The comparison plots clearly illustrated the effects of sampling rate changes on signal representation.